

OWLBOX

Verkabelung

Vollständige Hardware-Referenz:
Pinbelegung, 7"-DSI-Display, HiFiBerry, Netzwerk-Fallback.

Hardware-Aufbau Raspberry Pi 3B+
Schnelleinstieg: OwlBox-Schnellstart.pdf

Inhalt

Inhalt	2
1. Zielhardware	3
Anschluss: offizielles 7" Touch Display (DSI)	3
GPIO-Belegung im Überblick	4
2. HiFiBerry Amp2	6
Wichtig: vc4-kms-v3d braucht ,noaudio	6
Lautsprecher anschließen	7
3. RC522 RFID-Leser (Software-SPI auf freien GPIOs)	8
4. Taster (vor/zurück)	9
5. Dreh-Encoder mit Taster (KY-040)	10
Zweiter Dreh-Encoder für Helligkeit (KY-040)	10
6. Display-Hintergrundbeleuchtung	11
7. Fallback-Hotspot (WLAN-Recovery)	12
8. 7" Touch Display: kein Treiber-Setup nötig	13
Drehung um 180° (falls das Display auf dem Kopf verbaut ist)	13
9. Kiosk-Autostart (Chromium fullscreen, ohne Desktop-Umgebung)	15
10. Konfigurationsdatei (config.yaml)	16

1. Zielhardware

- Raspberry Pi 3B+
- HiFiBerry Amp2 (I2S-Verstärker-HAT, TAS5756M-Chip)
- RC522 RFID-Modul (SPI, 13,56 MHz)
- Offizielles Raspberry Pi 7" Touch Display (erste Generation - DSI-Flachbandkabel für Bild und Touch, plus 4 Jumperkabel für Strom/I2C, siehe unten). Ersetzt das früher hier dokumentierte 3,5"-SPI-Display (tft35a/MHS-35-Klon); dessen Anleitung ist noch in der Git-Historie dieser Datei zu finden, falls je wieder gebraucht.
- 2 Taster (vor/zurück)
- 1 Dreh-Encoder mit Druckschalter (Lautstärke/Play-Pause)
- 1 weiterer Dreh-Encoder ohne Taster (Helligkeit, s.u. - auf dieser Hardware aktuell ohne Wirkung, s. Kapitel 6)

Alle Pin-Angaben sind BCM-Nummerierung und entsprechen den Standardwerten in config/config.example.yaml. Wer andere Pins verdrahtet, passt einfach die gpio:/rfid:-Sektion in config.yaml an.

Hinweis: Referenzgrafiken liegen zusätzlich zu dieser PDF im Projekt unter docs/: owlbox-wiring-diagram.svg (Gesamtaufbau), owlbox-gpio-pinout.svg (kompletter 40-Pin-Header), owlbox-adapter-layout.svg (Platinenlayout-Vorschlag) und hat-wiring.html (kompletter Schaltplan, im Browser öffnen).

Anschluss: offizielles 7" Touch Display (DSI)

Anders als das frühere 3,5"-SPI-Display sitzt dieses Display **nicht** auf dem 40-Pin-Header - Bild und Touch laufen komplett über das mitgelieferte DSI-Flachbandkabel (eigener Steckplatz auf dem Pi, neben den HDMI-Buchsen). **Damit entfällt die frühere Steckplatz-Kollision mit dem HiFiBerry komplett** - der HiFiBerry sitzt normal direkt auf dem 40-Pin-Header, das Display hängt separat am DSI-Steckplatz.

Die kleine Adapter-/Power-Platine auf der Rückseite des Displays braucht trotzdem 4 Jumper-/Dupont-Kabel zum Pi, weil DSI selbst weder Strom noch die I2C-Leitung fürs Touch mitführt:

Display-Adapterplatine	Pi-Pin (BCM)	Zweck
5V	Pin 2 oder Pin 4	Stromversorgung
GND	Pin 6 (oder jeder andere GND-Pin)	Masse
SDA	Pin 3 (GPIO2)	I2C-Datenleitung (Touch-Controller)
SCL	Pin 5 (GPIO3)	I2C-Taktleitung (Touch-Controller)

Hinweis: Kein Konflikt mit dem HiFiBerry, obwohl GPIO2/3 dieselben Pins sind, die er für seine eigene I2C-Steuerung nutzt: I2C ist ein echter Mehrgeräte-Bus, mehrere Chips teilen sich Takt-/Datenleitung problemlos, solange sie unterschiedliche Adressen haben - der Touch-Controller des Displays und der HiFiBerry-Chip (Adresse 0x4d, per i2cdetect -y 1 bestätigt) sitzen auf unterschiedlichen Adressen.

Falls der HiFiBerry bereits vollflächig auf dem 40-Pin-Header aufgesteckt ist und die Pins dadurch von oben nicht mehr mit Dupont-Kabeln erreichbar sind: ein GPIO-Stacking-Header (Extra-Höhe, mit durchgeführten Pins) zwischen Pi und HiFiBerry löst das, ohne den HiFiBerry selbst umverkabeln zu müssen.

Hinweis: Kein Treiber-Installer nötig, aber ein eigener Overlay ist Pflicht: dtoverlay=vc4-kms-v3d allein (Standard seit Bookworm) reicht nicht - das aktiviert nur den generellen KMS-Grafiktreiber, kennt aber die Timings/das Panel dieses konkreten Displays nicht. Ohne den zusätzlichen, displayspezifischen Overlay dtoverlay=vc4-kms-dsi-7inch bindet der Treiber gar kein Panel, Bildschirm bleibt komplett schwarz. Details und Bild-/Touch-Rotation siehe Kapitel 8; install.sh trägt den Overlay automatisch ein.

GPIO-Belegung im Überblick

Funktion	BCM-Pin	Genutzt von
I2S BCLK	18	HiFiBerry
I2S LRCLK	19	HiFiBerry
I2S DIN	20	HiFiBerry
I2S DOUT	21	HiFiBerry
I2C SDA	2	HiFiBerry (Amp-Steuerung) und Display-Touch-Controller - gemeinsam am selben I2C-Bus, kein Konflikt (unterschiedliche Adressen), s.o.
I2C SCL	3	HiFiBerry (Amp-Steuerung) und Display-Touch-Controller, s.o.
SPI0 SCLK/MOSI/MISO/CE0/CE1	11 / 10 / 9 / 8 / 7	frei (das DSI-Display braucht kein SPI0 mehr; RC522 bleibt trotzdem auf Software-SPI, s. Kapitel 3)
RC522 SCK (Software-SPI)	4	RC522 (rfid.sck_pin)
RC522 MOSI (Software-SPI)	16	RC522 (rfid.mosi_pin)
RC522 MISO (Software-SPI)	15	RC522 (rfid.miso_pin)
RC522 SDA/CS (Software-SPI)	14	RC522 (rfid.cs_pin)
RC522 RST	26	RC522 (rfid.reset_pin)
Taster Weiter	5	Taster
Taster Zurück	6	Taster

Funktion	BCM-Pin	Genutzt von
Encoder CLK	1	Lautstärke-Encoder (17 wäre jetzt auch wieder frei, s. Kapitel 5)
Encoder DT	27	Lautstärke-Encoder
Encoder SW	22	Lautstärke-Encoder
Display-Backlight	-	läuft über Sysfs, kein GPIO mehr - Backlight-Dimmen aktuell nicht angeschlossen, s. Kapitel 6
Helligkeits-Encoder CLK	23	Helligkeits-Encoder (aktuell ohne Wirkung, s. Kapitel 6)
Helligkeits-Encoder DT	12	Helligkeits-Encoder (aktuell ohne Wirkung, s. Kapitel 6)

2. HiFiBerry Amp2

Der HiFiBerry belegt die I2S-Pins (BCM 18/19/20/21) sowie I2C (BCM 2/3) zur Verstärkersteuerung. In /boot/firmware/config.txt (bzw. /boot/config.txt auf älteren Images):

```
dtparam=audio=off
dtoverlay=hifiberry-dacplus
```

Hinweis: An echter Hardware bestätigt: Der Amp2 hat einen TAS5756M-Chip - dieselbe PCM512x-Chipfamilie wie die DAC+ Pro, ein komplett anderer Chip als der TAS5713 des älteren Amp/Amp+. dtoverlay=hifiberry-amp ist speziell für den TAS5713 und funktioniert mit dem Amp2 nicht: der Kernel spricht dann die falsche I2C-Adresse an, aplay -l zeigt „no soundcards found“ - äußert sich als Lautstärke, die sich nie ändert (bleibt bei 0). Zum Nachprüfen, welcher Chip verbaut ist: i2cdetect -y 1 - Adresse 0x4d antwortet beim TAS5756M/Amp2 (hifiberry-dacplus), Adresse 0x1b beim TAS5713 vom Amp/Amp+ (hifiberry-amp).

Nach einer Overlay-Änderung neu starten - dabei verschiebt sich meist auch die Kartennummer.

Danach mit aplay -l die Kartennummer der HiFiBerry ermitteln (z.B. „card 2: ...“) und mit amixer -c <Kartennummer> scontrols den Mixer-Namen prüfen - beides in config.yaml eintragen: audio.alsa_device ("hw:<Kartennummer>,0"), audio.mixer_control (Amp/Amp2 nutzen meist „Digital“, manche Boards „PCM“ oder „Master“) **und audio.mixer_card** (nur die Kartennummer, ohne hw:/,0).

Wichtig: Alle drei müssen zur selben Karte passen - install.sh trägt sie bei der automatischen Erkennung mittlerweile alle drei ein, aber wer das von Hand einträgt, vergisst leicht mixer_card: bleibt die auf ihrem Standardwert "0" stehen während die HiFiBerry tatsächlich auf einer anderen Kartennummer läuft, zielt jede Lautstärkeabfrage/-änderung ins Leere - äußert sich als „eingestellte Lautstärke wird nie gespeichert, zeigt immer 0“.

Wichtig: vc4-kms-v3d braucht ,noaudio

Der vc4-kms-v3d-Grafiktreiber-Overlay registriert standardmäßig zusätzlich eine eigene HDMI-Audio-ALSA-Karte (taucht in aplay -l als „card N: vc4hdmi“ auf), auch wenn HDMI-Audio in diesem Projekt nie genutzt wird (Anzeige läuft über DSI, Ton ausschließlich über den HiFiBerry).

Wichtig: An echter Hardware bestätigt, Ursache eines tagelangen „Wiedergabe knackt/fragmentiert trotz digital korrektem Signalweg“-Rätsels: Diese HDMI-Audio-Registrierung kollidiert offenbar mit dem I2S-Pfad des HiFiBerry (beide laufen letztlich über denselben VC4-I2S/Audio-Hardwareblock) - äußert sich als digital sauber ankommende, aber physisch knacksende/fragmentierte Wiedergabe, dazu wiederkehrende pcm512x-I2C-Fehler im Kernel-Log. Keins der naheliegenden Gegenmittel (Auto Mute an/aus, Bluetooth deaktivieren, Runtime-Power-Management-Sysfs-Override, selbst ein komplett frisches SD-Karten-Image) behebt das, weil keins davon die eigentliche Ursache anfasst.

Der Fix: ,noaudio an den Overlay anhängen, damit vc4-kms-v3d sich aus der Audio-Seite dieses gemeinsam genutzten Hardwareblocks komplett heraushält:

```
dtoverlay=vc4-kms-v3d,noaudio
```

install.sh trägt das automatisch so ein.

Wichtig: Zweite Falle, an echter Hardware bestätigt: Der Fix wirkt nur, wenn er die EINZIGE `dtoverlay=vc4-kms-v3d`-Zeile in der Datei ist. Ein frisches Raspberry Pi OS Bookworm-Image bringt in `config.txt` bereits eine eigene, unkommentierte `dtoverlay=vc4-kms-v3d`-Zeile mit (ohne `,noaudio`). Anders als `dtoverlay=-`-Zeilen sind `dtoverlay=`-Zeilen KEINE Key/Value-Overrides - jede einzelne wendet den Overlay unabhängig an. Stehen beide Zeilen in der Datei, registriert die erste trotzdem ihre eigene `vc4hdmi`-Karte, und das Knacksen bleibt bestehen. Kontrolle: `grep -n dtoverlay=vc4-kms-v3d config.txt` sollte genau einen Treffer zeigen (den mit `,noaudio`); `aplay -l` sollte keine `vc4hdmi`-Karte mehr auflisten. `install.sh` passt die vorbestehende Zeile jetzt automatisch direkt an Ort und Stelle an (statt sie zu löschen und eine eigene Kopie ans Dateiende anzuhängen) - auf einer schon länger laufenden Installation reicht dafür ein erneutes `sudo owlbox-install` plus Neustart. Wichtig: dafür wirklich `owlbox-install` verwenden (ein stabiler Befehl, den das Skript bei seinem ersten erfolgreichen Durchlauf selbst unter `/usr/local/bin` anlegt), nicht `cd owlbox && sudo ./scripts/install.sh` - `cd owlbox` von innerhalb eines bereits ausgecheckten Repos landet nicht im Repo-Root, sondern eine Ebene zu tief im gleichnamigen Python-Paket-Unterverzeichnis, und `./scripts/install.sh` meldet dann nur „command not found“, ohne dass irgendetwas vom Skript tatsächlich läuft.

Wichtig: Dritte Falle, ebenfalls an echter Hardware bestätigt: `dtoverlay=audio=on` muss aus demselben Grund verschwinden, nicht nur auskommentiert oder von einem späteren `dtoverlay=audio=off` "überschrieben" werden. Ein frisches Bookworm-Image bringt standardmäßig eine eigene, unkommentierte `dtoverlay=audio=on`-Zeile mit. Die naheliegende Annahme - `dtoverlay=-`-Zeilen seien Key/Value-Overrides, bei denen die letzte Zeile gewinnt, also würde `install.sh`'s eigenes, weiter unten stehendes `dtoverlay=audio=off` automatisch siegen - hat sich an echter Hardware NICHT zuverlässig bestätigt: die onboard „bcm2835 Headphones“-ALSA-Karte tauchte trotz korrekt zuletzt stehendem `dtoverlay=audio=off` über mehrere Neustarts hinweg immer wieder in `aplay -l` auf. `install.sh` kommentiert seit dieser Erkenntnis die vorbestehende `dtoverlay=audio=on`-Zeile automatisch direkt an Ort und Stelle aus (`#dtoverlay=audio=on`), statt sich auf Override-Semantik zu verlassen - derselbe `sudo owlbox-install` plus Neustart wie oben behebt beides in einem Rutsch.

Lautsprecher anschließen

Der HiFiBerry Amp2 hat dafür keine Stecker (kein Cinch/Klinke), sondern zwei 2-polige Federklemmen direkt auf der Platine (eine pro Kanal, jeweils +/-). Angeschlossen wird ganz normales 2-adriges Lautsprecherkabel:

- Querschnitt $\geq 0,75 \text{ mm}^2$ (AWG 18) reicht für 15W/4Ω locker; bei längeren Kabelwegen (> 3-5m) eher 1,0-1,5 mm² nehmen.
- Enden abisolieren (~10mm); bei feindrähtiger Litze verzinnen oder Aderendhülsen verwenden, damit die Federklemme sauber greift.
- Polarität an beiden Lautsprechern konsistent anschließen (+ zu + und Minus zu Minus) - sonst laufen sie gegenphasig und Bass/Stereo-Ortung leiden.
- Am Lautsprecher selbst hängt der Anschluss vom jeweiligen Modell ab (blanker Draht, Flachsteckhülsen/Bananas oder Lötflanschen).

3. RC522 RFID-Leser (Software-SPI auf freien GPIOs)

Hinweis: Anders als in den meisten RC522-Anleitungen im Netz hängt der RC522 hier NICHT an einem der beiden Hardware-SPI-Busse des Pi, sondern an vier per Software angesteuerten GPIOs. Grund: SPI1 liegt auf GPIO18-21, exakt den Pins, die der HiFiBerry für I2S-Ton braucht - SPI0 ist inzwischen zwar frei (das offizielle 7"-DSI-Touch-Display beansprucht es anders als das frühere SPI-Display nicht mehr), die RC522-Verdrahtung bleibt hier aber trotzdem auf Software-SPI, um nicht mehr als nötig gleichzeitig umzustellen. Der RC522 hat keine Mindesttakttrate, Software-SPI funktioniert daher zuverlässig, nur etwas langsamer als Hardware-SPI - für einen Chip-Scan völlig ausreichend.

RC522-Pin	Raspberry Pi	Config-Feld
VCC	3.3V (nicht 5V!)	-
GND	GND	-
RST	GPIO26	rfid.reset_pin
SDA (CS)	GPIO14	rfid.cs_pin
SCK	GPIO4	rfid.sck_pin
MOSI	GPIO16	rfid.mosi_pin
MISO	GPIO15	rfid.miso_pin
IRQ	nicht verbunden	-

Alle vier GPIOs (4/14/15/16) sind sonst von nichts in diesem Projekt belegt. SPI selbst muss trotzdem aktiviert bleiben, weil der RC522 Software-SPI über I2C braucht (macht `scripts/install.sh` bereits via `raspi-config nonint do_spi 0`, alternativ `sudo raspi-config` → Interface Options → SPI).

Hinweis: Historischer Hintergrund zum Lautstärke-Encoder auf GPIO1 statt GPIO17: Das frühere SPI-Display beanspruchte zusätzlich zu SPI0/CE0/CE1 auch GPIO17 als Interrupt-Pin („pendown“) für den (nie verdrahteten) Touch-Controller - fest im damaligen Overlay einprogrammiert, unabhängig davon, ob Touch physisch angeschlossen war. Deshalb liegt der Lautstärke-Encoder-CLK auf GPIO1 (ID_SC, konventionell für ein HAT-ID-EEPROM reserviert, hier aber echt frei, da der HiFiBerry ohnehin per manueller `dtoverlay`-Zeile statt EEPROM-Erkennung konfiguriert wird). Mit dem neuen DSI-Display ist GPIO17 jetzt wieder frei - die Verkabelung bleibt hier trotzdem auf GPIO1, um nicht ohne Grund vom dokumentierten Standard abzuweichen; wer umverkabeln will, kann `gpio.encoder_clk` in `config.yaml` frei auf GPIO17 umstellen.

An echter Hardware bestätigt: Sowohl das Software-SPI als auch der RC522-Reset-Pin laufen über I2C (`owlbox/rfid/igpio_compat.py`), nicht über `RPi.GPIO` - obwohl die `mfr522`-Bibliothek intern eigentlich fest auf `RPi.GPIO` setzt (wird per `unittest.mock.patch` umgeleitet). Grund: `gpiozero` (Taster/Encoder) braucht auf aktuellen Kernen zwingend I2C, weil `RPi.GPIO`s eigene Kantenerkennung dort mit „Failed to add edge detection“ abbricht - `RPi.GPIO` zeigte in diesem Prozess außerdem selbst für unbenutzte Pins sofort „already in use“-Warnungen. Deshalb läuft die komplette GPIO-Ansteuerung dieses Projekts konsistent über I2C, nirgends mehr über `RPi.GPIO`.

4. Taster (vor/zurück)

Als Taster kommen Cherry-MX-Switches (3-Pin-Variante) zum Einsatz - elektrisch ganz normale Momentary-Schalter (schließt nur beim Drücken, öffnet sonst).

- Von den 3 Pins sind nur die beiden Metall-Pins die elektrischen Kontakte (diagonal gegenüberliegend); der dritte, meist aus Kunststoff, ist nur ein mechanischer Halteclip ohne elektrische Funktion und muss nirgends angeschlossen werden.
- Jeweils einer der beiden Metall-Pins an GPIO, der andere an GND. Kein externer Widerstand nötig, der interne Pull-up wird von gpiozero aktiviert.
- Da Cherry-MX-Switches (anders als billige Blechtaster) sehr sauber prellen, reicht die Standard-Entprellzeit (`gpio.bounce_time`, 50ms) komfortabel aus.

Funktion	BCM-Pin
Zurück	6
Weiter	5

Kurz drücken springt zum vorherigen/nächsten Track. Gedrückt halten (länger als `gpio.seek_hold_seconds`, Standard 0,4s) spult stattdessen im aktuellen Track vor/zurück, in Schritten von `gpio.seek_step_seconds` (Standard 10s) - kein Trackwechsel, solange gehalten wird.

5. Dreh-Encoder mit Taster (KY-040)

Encoder-Pin	Raspberry Pi
CLK	GPIO1 (nicht 17, siehe Kapitel 3)
DT	GPIO27
SW	GPIO22
+	3.3V
GND	GND

Das KY-040-Modul bringt eigene Pull-up-Widerstände für CLK/DT mit; die zusätzlich von gpiozero aktivierten Pull-ups des Pi stören dabei nicht (einfach parallel). Für den Taster (SW) hat das Modul in der Regel keinen eigenen Pull-up - das übernimmt `gpiozero.Button(pull_up=True)` im Code, hier also nichts weiter nötig.

Drehen ändert die Lautstärke (Schrittweite `audio.volume_step`), Drücken schaltet Play/Pause um. Ein langer Druck (`gpio.shutdown_hold_seconds`, Standard 4s) fährt den Pi sicher herunter - praktisch für ein Kindergerät ohne Zugriff auf ein Terminal. Auf 0 setzen, um das abzuschalten.

Zweiter Dreh-Encoder für Helligkeit (KY-040)

Gehört zum Standardaufbau - dasselbe KY-040-Modul, diesmal ohne den Taster zu verdrahten (kein eigener Klick, nur Drehen). **Auf dieser Hardware aktuell ohne Wirkung** (siehe Kapitel 6) - der Encoder selbst kann so lange unverdrahtet bleiben.

Encoder-Pin	Raspberry Pi
CLK	GPIO23
DT	GPIO12
+	3.3V
GND	GND

Drehen ändert die Helligkeit (Schrittweite `gpio.brightness_step`, Standard 5%), sofort und rein manuell - es gibt kein automatisches Dimmen.

Hinweis: Damit Shutdown/Neustart (auch über die Web-UI oder einen Funktions-Chip) sowie der Update-Button auf der Info-Seite ohne Passwortabfrage funktionieren, braucht der Service-User owlbox passwortloses sudo dafür. `install.sh` richtet das automatisch ein (`/etc/sudoers.d/owlbox`, syntaxgeprüft vor dem Einspielen) - hier nur zur Referenz:

```
owlbox ALL=(ALL) NOPASSWD: /sbin/shutdown, /usr/bin/nmcli, \
/usr/bin/systemctl restart --no-block owlbox
```

Wichtig: Die Argumente müssen exakt wie oben dastehen (inklusive `--no-block`) - sudo vergleicht die komplette Befehlszeile. Fehlt `--no-block`, meldet der Update-Button auf der Info-Seite beim Neustart „sudo: a password is required“.

6. Display-Hintergrundbeleuchtung

Hinweis: Wichtiger Unterschied zum alten Display: Dieses Display hat keine per GPIO/PWM ansteuerbare LED-Leitung wie das alte SPI-Display - die Helligkeit wird stattdessen intern über eine Linux-Backlight-Sysfs-Schnittstelle geregelt (/sys/class/backlight/.../brightness), angesteuert vom Power-Chip auf der Display-Adapterplatine selbst. Es gibt daher keine eigene Verkabelung mehr für diesen Punkt - kein Transistor, kein MOSFET, keine LED-Leitung, kein gpio.backlight_pin.

Die GPIO13-Transistor-Schaltung und gpio.backlight_pin aus der alten Verkabelung entfallen damit ersatzlos - **das Backlight-Dimmen über den zweiten Dreh-Encoder ist auf dieser Hardware aktuell nicht angeschlossen**, das müsste in owlbox/controls/gpio_controls.py erst auf die Sysfs-Schnittstelle umgestellt werden. Der zweite Encoder (Kapitel 5) kann so lange unverdrahtet bleiben - jede Helligkeitsänderung über die Web-Oberfläche blendet den neuen Wert zwar kurz auf dem Display ein, die eigentliche Backlight-Steuerung greift aber noch nicht.

7. Fallback-Hotspot (WLAN-Recovery)

Ist WLAN eingeschaltet, aber für `network.hotspot_after_seconds` (Standard 60s) mit keinem Netzwerk verbunden - z.B. weil das Heimnetz sein Passwort geändert hat oder der Pi an einen neuen Ort umgezogen ist - macht der Pi automatisch seinen eigenen Access Point auf (`nmcli device wifi hotspot`), statt komplett unerreichbar zu bleiben.

SSID und Passwort (aus `config.yaml` unter `network:`, Standard „OwlBox-Setup“ / „owlbox-setup“) werden auf dem Kiosk-Display und im Admin-Bereich angezeigt, inklusive der URL, unter der die Einstellungen im Hotspot erreichbar sind (normalerweise `http://10.42.0.1:5000/admin` - NetworkManagers Standard-Adresse für einen geteilten Access Point).

Ablauf: mit einem Laptop/Handy in dieses WLAN einwählen, die angezeigte URL öffnen, unter Einstellungen → WLAN das eigentliche Netzwerk auswählen/verbinden. Sobald das klappt, beendet der Pi den Hotspot von selbst wieder.

Wichtig: Das Standardpasswort „owlbox-setup“ steht so im Quellcode und ist damit öffentlich bekannt - für den Einsatz in einer Umgebung, in der Fremde in Funkreichweite kommen könnten, unbedingt in `config.yaml` ein eigenes Passwort setzen.

8. 7" Touch Display: kein Treiber-Setup nötig

Im Gegensatz zum früheren 3,5"-SPI-Display (das einen virtuellen-HDMI-Trick, fbcp und den alten Legacy-Grafiktreiber brauchte, um überhaupt ein Bild zu zeigen) ist das offizielle 7"-Display an einem normalen Raspberry Pi OS Bookworm-Image komplett plug-and-play: Firmware erkennt es automatisch über das DSI-Kabel, keine dtoverlay=-Zeile, kein Treiber-Installer, kein Extra-Paket. Empfohlenes Basis-Image bleibt trotzdem Raspberry Pi OS Lite, 64-bit (ohne Desktop-Umgebung) - der Kiosk startet X selbst nur für Chromium (siehe Kapitel 9), eine mitinstallierte Desktop-Umgebung (lightdm, LXDE) würde beim Boot nur unnötig Zeit kosten. Wichtig ist nur: der moderne KMS-Grafiktreiber (vc4-kms-v3d) bleibt aktiv (Bookworm-Standard) - er wurde beim alten Display extra deaktiviert, das ist mit diesem Display nicht mehr nötig und würde die GPU-Beschleunigung sogar wieder kosten.

Hinweis: Ein eigener Overlay ist trotzdem Pflicht: dtoverlay=vc4-kms-v3d,noaudio allein aktiviert nur den generellen KMS-Treiber, kennt aber die Timings dieses konkreten Panels nicht - zusätzlich braucht es dtoverlay=vc4-kms-dsi-7inch. Ohne diesen zweiten Overlay bindet der Treiber gar kein Panel (dmesg zeigt „[drm] Cannot find any crtc or sizes“, Bildschirm bleibt komplett schwarz, kein Fehler sonst irgendwo sichtbar). install.sh trägt beide Zeilen automatisch in /boot/firmware/config.txt ein:

```
dtoverlay=vc4-kms-v3d,noaudio
dtoverlay=vc4-kms-dsi-7inch
```

Drehung um 180° (falls das Display auf dem Kopf verbaut ist)

Zwei verschiedene Stellschrauben für zwei verschiedene Dinge, an echter Hardware mühsam herausgefunden:

- **Bild selbst:** Der vc4-kms-dsi-7inch-Overlay hat KEINEN rotate=-Parameter (/boot/firmware/overlays/README listet nur sizeX/sizeY/invX/invY/swapXY/disable_touch/dsi0 - ein versuchsweise angehängtes rotate=180 wird stillschweigend ignoriert). Die Bild-Rotation läuft stattdessen über einen Kernel-Boot-Parameter in cmdline.txt (nicht config.txt!): ans Ende der einzeiligen Datei anhängen.
- **Touch-Koordinaten:** KEINE zusätzlichen invX/invY-Parameter am Overlay setzen. Sobald das Bild über den cmdline.txt-Parameter gedreht ist, korrigiert X11/libinput die Touch-Koordinaten am gedrehten Ausgang bereits von sich aus passend mit - zusätzlich gesetztes invX,invY dreht dann nochmal drüber und zeigt sich als auf beiden Achsen spiegelverkehrter Touch relativ zum (korrekt gedrehten) Bild.

```
video=DSI-1:800x480@60,rotate=180
```

Wichtig: install.sh trägt das automatisch ein. Wichtig: nicht über xrandr oder display_lcd_rotate versuchen - an echter Hardware bestätigt: xrandr --output DSI-1 --rotate inverted wird zwar anstandslos angenommen (xrandr --query zeigt danach „inverted“), das Panel zeichnet aber nie tatsächlich neu, selbst nach einem erzwungenen --off/--auto-Modeset. Der ältere Parameter display_lcd_rotate/lcd_rotate ist unter KMS ebenfalls wirkungslos.

Einfach dtoverlay=vc4-kms-dsi-7inch ohne weitere Parameter reicht, der Touch-Controller ist ohnehin Teil desselben Overlays, kein separater rpi-ft5406-Eintrag nötig.

9. Kiosk-Autostart (Chromium fullscreen, ohne Desktop-Umgebung)

Da die Basis „Lite“ keine Desktop-Umgebung mitbringt, gibt es auch kein lightdm/LXDE, in das sich der Kiosk einhängen könnte. Stattdessen startet ein eigener systemd-Dienst (owlbox-kiosk.service) X direkt selbst (per startx), übernimmt dafür tty1 und lässt scripts/kiosk.sh (das Chromium im Kiosk-Modus startet) als einzigen „Client“ laufen - keine Fensterleiste, kein Dateimanager-Desktop, kein Panel. Mit aktivem KMS-Treiber findet X's eigener, automatisch gewählter „modesetting“-Treiber /dev/dri/card0 von selbst - keine eigene Xorg-Konfiguration nötig (anders als beim alten Display, das X explizit auf einen Framebuffer-Treiber zwingen musste).

```
# X ohne Display-Manager erlauben:
cat > /etc/X11/Xwrapper.config <<'EOF'
allowed_users=anybody
needs_root_rights=yes
EOF

sudo cp /opt/owlbox/systemd/owlbox-kiosk.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now owlbox-kiosk.service
```

Hinweis: owlbox-kiosk.service bringt Conflicts=getty@tty1.service schon mit, muss also getty@tty1.service nicht extra deaktiviert bekommen - läuft aber sauberer, wenn man es trotzdem tut (sudo systemctl disable getty@tty1.service), damit dort kein ungenutzter Login-Prompt mehr mitstartet.

Läuft doch eine volle Desktop-Umgebung (z.B. weil bewusst die volle Variante statt Lite geflasht wurde), lässt sich der Kiosk alternativ ganz klassisch über deren Autostart-Datei einhängen: ~/.config/lxsession/LXDE-pi/autostart um die Zeile @/opt/owlbox/scripts/kiosk.sh ergänzen.

Wichtig: An echter Hardware bestätigt: owlbox-kiosk.service läuft bewusst mit niedrigerer CPU-/IO-Priorität als owlbox.service (Nice=15, IOSchedulingClass=best-effort, IOSchedulingPriority=7). Grund: Chromium läuft auf dieser Hardware komplett softwaregerendert (keine GPU-Beschleunigung verfügbar) - ohne Prioritätsdifferenz konkurrieren Chromium und mpv (läuft in owlbox.service) mit exakt gleicher Priorität um die knappe CPU eines Pi 3B+. Das war die eigentliche Ursache eines "Wiedergabe knackt durchgehend"-Rätsels, das auch nach dem Beheben aller config.txt-Probleme (Kapitel 2) und dem Entfernen unnötiger Subprozess-Aufrufe aus dem App-Code bestehen blieb: ein reiner config.txt-Test direkt nach frischer Installation plus aplay/mpv im Terminal (ganz ohne laufenden Kiosk) spielte sauber ab, derselbe Aufbau mit laufendem Kiosk knackste weiterhin. Ein positiver Nice-Wert braucht keine besonderen Rechte - install.sh trägt das automatisch ein und startet owlbox-kiosk.service bei Bedarf neu, damit die neue Priorität auch ohne kompletten Neustart greift.

10. Konfigurationsdatei (config.yaml)

Alle in dieser Anleitung genannten Werte (Pins, Schrittweiten, Zeiten) stehen gesammelt in config/config.yaml (aus config/config.example.yaml kopieren). Die wichtigsten Abschnitte:

Abschnitt	Wichtigste Werte
audio:	alsa_device, mixer_control, default_volume, volume_step, chime_enabled
rfid:	reader, sck_pin, mosi_pin, miso_pin, cs_pin, reset_pin, poll_interval
gpio:	button_next/prev, encoder_clk/dt/switch, backlight_pin, brightness_encoder_clk/dt, shutdown_hold_seconds, seek_hold_seconds
playback:	restart_track_after_seconds, position_save_interval, auto_sleep_minutes, sleep_fade_seconds
network:	hotspot_ssid, hotspot_password, hotspot_after_seconds
web:	host, port, secret_key

Hinweis: config/config.yaml ist in .gitignore eingetragen, damit lokale, gerätespezifische Werte (insb. das Session-Secret) nie versehentlich committet werden.